

Inhaltsverzeichnis

```
#include <Arduino.h>
#include "config.h"
#include "modbus.h"
#include "ad_da.h"
void setup() {
    setup_modbus();
    setup_ad_da();
} // setup
void loop() {
    WorkModbus();
    WorkAdDa();
} // loop
// ----- ad_da -----
#include <Wire.h>
#include <SPI.h>
#include "modbus.h"
#include "ad_da.h"
SPISettings SPISettings_AD7682(400000, MSBFIRST, SPI_MODE0);
SPISettings SPISettings_DAC8565(400000, MSBFIRST, SPI_MODE2);
uint16_t getAD7682(int aCS, uint8_t aKanal, uint16_t aRange){
    uint8_t c[2];
    digitalWrite(aCS, LOW);
    delayMicroseconds(1);
    aRange |= (aKanal << 9); // << (7+2)
    c[0]=highByte(aRange);
    c[1]=lowByte(aRange);
    SPI.transfer(c, 2);
    delayMicroseconds(1);
    digitalWrite(aCS, HIGH);
    delayMicroseconds(1);
    return (c[0]<<8) | c[1];
} // getAD7682
uint16_t calc_miw(uint16_t mw, uint16_t *miws, int *imiw, int NMIW){
    miws[*imiw]=mw;
    (*imiw)++;
    if (*imiw>=NMIW) *imiw=0;
    int miw_sum=miws[0];
    for (int n=1; n<NMIW; n++) miw_sum+=miws[n];
    return miw_sum / NMIW;
} // calc_miw
void ReadADCs(){
#define AD_MODE (0x2000 | 0x1C00 | 0x0008 | 0x0001) << 2 // CFG update | Unipol to GND | low BW,
    SPI.beginTransaction(SPISettings_AD7682);
    static uint16_t miws0[NMIW_AD]={}, miws1[NMIW_AD]={}, miws2[NMIW_AD]={}, miws3[NMIW_AD]={};
    static int imiw0=0, imiw1=0, imiw2=0, imiw3=0;
    // bei der Abfrage wird Kanal+1 angefragt und Kanal-1 geliefert:
    reg[REG_AD0+0]=getAD7682(PIN_CS_AD,2, AD_MODE); // cs2, CH0=Urc1, sigle 0..4V
    reg[REG_ADMIW+0]=calc_miw(reg[REG_AD0+0], miws0,&imiw0, NMIW_AD);
    reg[REG_AD0+1]=getAD7682(PIN_CS_AD,3, AD_MODE); // cs2, CH1=Urc2, sigle 0..4V
    reg[REG_ADMIW+1]=calc_miw(reg[REG_AD0+1], miws1,&imiw1, NMIW_AD);
    reg[REG_AD0+2]=getAD7682(PIN_CS_AD,0, AD_MODE); // cs2, CH2=Urb1, sigle 0..4V
    reg[REG_ADMIW+2]=calc_miw(reg[REG_AD0+2], miws2,&imiw2, NMIW_AD);
    reg[REG_AD0+3]=getAD7682(PIN_CS_AD,1, AD_MODE); // cs2, CH3=Urb2, sigle 0..4V
```

```

    reg[REG_ADMIW+3]=calc_miw(reg[REG_AD0+3], miws3,&imiw3, NMIW_AD);
    SPI.endTransaction();
} // ReadADCs
void setDAC8565(int aCS, int aKanal, uint16_t aWert){ // 4x DA 0..2.5V
    uint8_t c[3];
    digitalWrite(aCS, LOW);
    delayMicroseconds(1);
    c[0]=(aKanal << 1) | 0x10;
    c[1]=highByte(aWert);
    c[2]=lowByte(aWert);
    SPI.transfer(c, 3);
    delayMicroseconds(1);
    digitalWrite(aCS, HIGH);
    delayMicroseconds(1);
} // setDAC8565
void WriteDAs(){
    SPI.beginTransaction(SPISettings_DAC8565);
    setDAC8565(PIN_CS_DA, 0, reg[REG_DA0+0]); // UC
    setDAC8565(PIN_CS_DA, 1, reg[REG_DA0+1]); // UB
    // ch2, ch3: NC
    SPI.endTransaction();
} // WriteDAs

void setup_ad_da(){
    SPI.begin();
    pinMode(PIN_CS_AD, OUTPUT);
    pinMode(PIN_CS_DA, OUTPUT);
    digitalWrite(PIN_CS_AD, HIGH);
    digitalWrite(PIN_CS_DA, HIGH);
} // setup_ad_da
void WorkAdDa(){
    ReadADCs();
    WriteDAs();
} // WorkAdDa
// ----- ad_da -----
#ifdef __AD_DA_H__
#define __AD_DA_H__
#include "config.h"
extern "C"{
void setup_ad_da();
void WorkAdDa();
} // extern "C"
#endif /* ifndef */
// ----- config -----
#ifdef __CONFIG_H__
#define __CONFIG_H__
#include <Arduino.h>
#define SerialUSB Serial
#define MODBUS_ADR 1
#define PIN_CS_DA 5
#define PIN_CS_AD 17
#define PIN_MB_LED 22
#define REG_DA0      0 // 4 Register, Uc, Ub, NC,NC
#define REG_AD0      4 // 4 Register, Urc1, Urc2, Urb1, Urb2
#define REG_ADMIW     8 // 4 Register, wie REG_AD0 als Mittelwerte

```

```

#define MAX_REG          13 // 12=letzter Register
#define NMIW_AD         10 // Anzahl für Mittelwert
#endif /* ifndef */
// ----- modbus -----
#include "modbus.h"
uint16_t reg[MAX_REG]={0};
void setup_modbus(){
    SerialUSB.begin(115200);
    pinMode(PIN_MB_LED, OUTPUT);
    digitalWrite(PIN_MB_LED, LOW);
} // setup_modbus
char* max(char *a, char*b){
    if (a<b) return b; else return a;
} // min
uint32_t MBword(char *a){
    return (*a << 8) | *(a+1);
}
void wordMB(uint32_t w, char *a){
    *a=w >> 8;
    a++;
    *a=w & 0xFF;
}
bool ReadByteUSB(char* a){
    if (SerialUSB.available()){
        *a=SerialUSB.read();
        return true;
    }
    return false;
} // ReadByteUSB
bool Sz(char a){
    return ((a>=1) && (a<=6)) || (a==8) || (a==15) || (a==16));
}
// ----- crc modbus -----
// ----- Modbus Checksum -----
uint16_t modbus_crc(char *a, int len){
    // Table of CRC values for high-order byte:
    static const unsigned char Modbus_auchCRCHi[256] = {
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40,0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,
        0x00,0xC1,0x81,0x40,0x01,0xC0,0x80,0x41,0x01,0xC0,0x80,0x41,0x00,0xC1,0x81,0x40};
    // Table of CRC values for low-order byte:
    static const unsigned char Modbus_auchCRCLo[256] = {

```

```

0x00,0xC0,0xC1,0x01,0xC3,0x03,0x02,0xC2,0xC6,0x06,0x07,0xC7,0x05,0xC5,0xC4,0x04,
0xCC,0x0C,0x0D,0xCD,0x0F,0xCF,0xCE,0x0E,0x0A,0xCA,0xCB,0x0B,0xC9,0x09,0x08,0xC8,
0xD8,0x18,0x19,0xD9,0x1B,0xDB,0xDA,0x1A,0x1E,0xDE,0xDF,0x1F,0xDD,0x1D,0x1C,0xDC,
0x14,0xD4,0xD5,0x15,0xD7,0x17,0x16,0xD6,0xD2,0x12,0x13,0xD3,0x11,0xD1,0xD0,0x10,
0xF0,0x30,0x31,0xF1,0x33,0xF3,0xF2,0x32,0x36,0xF6,0xF7,0x37,0xF5,0x35,0x34,0xF4,
0x3C,0xFC,0xFD,0x3D,0xFF,0x3F,0x3E,0xFE,0xFA,0x3A,0x3B,0xFB,0x39,0xF9,0xF8,0x38,
0x28,0xE8,0xE9,0x29,0xEB,0x2B,0x2A,0xEA,0xEE,0x2E,0x2F,0xEF,0x2D,0xED,0xEC,0x2C,
0xE4,0x24,0x25,0xE5,0x27,0xE7,0xE6,0x26,0x22,0xE2,0xE3,0x23,0xE1,0x21,0x20,0xE0,
0xA0,0x60,0x61,0xA1,0x63,0xA3,0xA2,0x62,0x66,0xA6,0xA7,0x67,0xA5,0x65,0x64,0xA4,
0x6C,0xAC,0xAD,0x6D,0xAF,0x6F,0x6E,0xAE,0xAA,0x6A,0x6B,0xAB,0x69,0xA9,0xA8,0x68,
0x78,0xB8,0xB9,0x79,0xBB,0x7B,0x7A,0xBA,0xBE,0x7E,0x7F,0xBF,0x7D,0xBD,0xBC,0x7C,
0xB4,0x74,0x75,0xB5,0x77,0xB7,0xB6,0x76,0x72,0xB2,0xB3,0x73,0xB1,0x71,0x70,0xB0,
0x50,0x90,0x91,0x51,0x93,0x53,0x52,0x92,0x96,0x56,0x57,0x97,0x55,0x95,0x94,0x54,
0x9C,0x5C,0x5D,0x9D,0x5F,0x9F,0x9E,0x5E,0x5A,0x9A,0x9B,0x5B,0x99,0x59,0x58,0x98,
0x88,0x48,0x49,0x89,0x4B,0x8B,0x8A,0x4A,0x4E,0x8E,0x8F,0x4F,0x8D,0x4D,0x4C,0x8C,
0x44,0x84,0x85,0x45,0x87,0x47,0x46,0x86,0x82,0x42,0x43,0x83,0x41,0x81,0x80,0x40};
unsigned char uchCRCHi=0xFF, uchCRCLo=0xFF;
int uIndex,p;
for (p=0; p<len; p++) {
    uIndex=(uchCRCHi ^ a[p]);
    uchCRCHi=(uchCRCLo ^ Modbus_auchCRCHi[uIndex]);
    uchCRCLo=Modbus_auchCRCLo[uIndex];
}
return ((uchCRCHi<<8)|uchCRCLo);
} // modbus_crc
bool Modbus_chkCRC(char *a, int len){
    return (MBword(&a[len-2])==modbus_crc(a, len-2));
} // Modbus_chkCRC
void delCahrs(char *a, char **pa, int anz){
    *pa=max(a, *pa-anz);
    for (char* p=a; p<*pa; p++) *p=*(p+anz);
} // delChars
void setReg(uint16_t aReg, uint16_t aWert){
    if ((aReg<REG_AD0) && (reg[aReg]!=aWert)){
        reg[aReg]=aWert;
    }
} // setReg
void WorkModbus(){
    static char a[255], *pa=a;
    static int led_timer=0;
    char c;
    uint16_t SollLen,ServerAdr,RegAdr,Nreg,Nbyte,Ikanal;
    if (ReadByteUSB(&c)){
        *pa++=c;
        while ((pa-a>=8) && !Sz(a[1])) delCahrs(a,&pa,1);
        if (pa-a>=8){
            if ((a[1]==15) || (a[1]==16)) SollLen=a[6]+9; else SollLen=8;
            if (pa-a>=SollLen){
                if (Modbus_chkCRC(a, SollLen)) {
                    digitalWrite(PIN_MB_LED, HIGH);
                    led_timer=100;
                    ServerAdr=a[0];
                    if ((ServerAdr==0) || (ServerAdr==MODBUS_ADR)){
                        char r[255], *pr=r;
                        RegAdr=MBword(&a[2]);

```

```

Nreg=MBword(&a[4]);
switch (a[1]){
    case 1:
    case 2: // DI
        Nbyte=(Nreg+7) / 8;
        *pr+=a[0];
        *pr+=a[1];
        *pr+=Nbyte;
        for (int n=0; n<Nbyte; n++) *pr+=0;
        break;
    case 3:
    case 4: // WI
        Nreg=min(Nreg, (uint16_t)124);
        Nbyte=Nreg*2;
        Ikanal=RegAdr;
        *pr+=a[0];
        *pr+=a[1];
        *pr+=Nbyte;
        for (int n=0; n<Nreg; n++){
            if (Ikanal<MAX_REG) wordMB(reg[Ikanal], pr); else wordMB(0, pr);
            Ikanal++;
            pr+=2;
        }
        break;
    case 5: // D0 single
        // RegAdr, Bitwert in Nreg
        for (int n=0; n<6; n++) *pr+=a[n];
        break;
    case 6: // W0 single
        setReg(RegAdr, Nreg); // Nr, Wert
        for (int n=0; n<6; n++) *pr+=a[n];
        break;
    case 8:
        for (int n=0; n<6; n++) *pr+=a[n];
        break;
    case 15: // D0 multiple
        for (int n=0; n<6; n++) *pr+=a[n];
        break;
    case 16: // W0 multiple
        Nreg=min(Nreg, (uint16_t)(a[6] / 2));
        Ikanal=RegAdr;
        pa=&a[7];
        for (int n=0; n<Nreg; n++){
            setReg(Ikanal, MBword(pa)); // Nr, Wert
            Ikanal++;
            pa+=2;
        }
        for (int n=0; n<6; n++) *pr+=a[n];
        break;
}
if ((ServerAdr>0) && (pr>r)){ // 0=Broadcast: bearbeiten und nicht antworten
    wordMB(modbus_crc(r, pr-r), pr);
    pr+=2;
    SerialUSB.write(r, pr-r);
}

```

```

        } // if ServerAdr ...
        delCahrs(a,&pa,SollLen);
    } else delCahrs(a,&pa,1);
    } // if pa-a>=SollLen
    } // if pa-a>=8
} // if ReadByteUSB(&c)
if (led_timer){
    led_timer--;
    if (!led_timer) digitalWrite(PIN_MB_LED, LOW);
}
} // WorkModbus
// ----- modbus -----
#ifndef __MODBUS_H__
#define __MODBUS_H__
#include "config.h"
extern "C"{
extern uint16_t reg[MAX_REG];
void setup_modbus();
void WorkModbus();
} // extern "C"
#endif /* ifndef */

```